EXERCISES 03

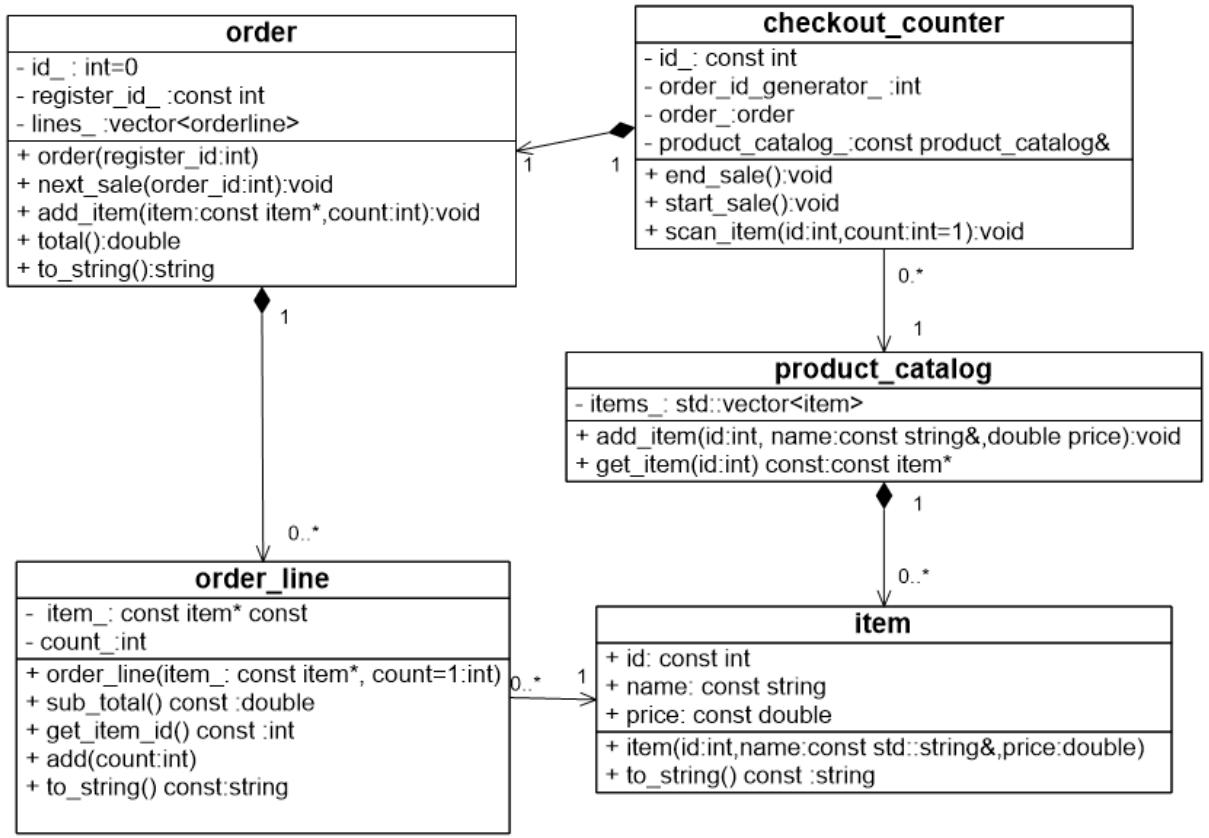
AARHUS
UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER
ENGINEERING

17 AUGUST 2023

MICHEL VEDEL HOWARD
ASSISTANT PROFESSOR

EXERCISE 1

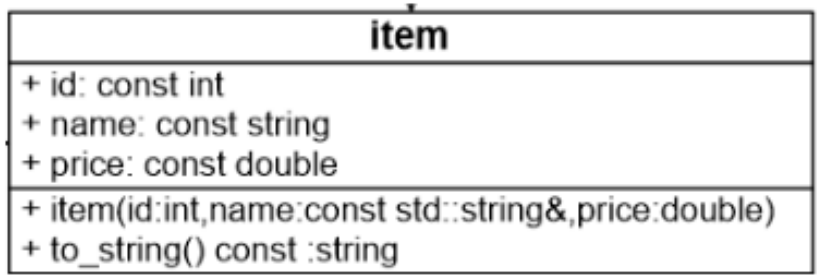
The exercise is concerned with implementing a sales system model



EXERCISE 1A

This is broken down into sub exercises

Implement the **struct**



The members id_, name, price are all public but should be immutable. And therefore, they are all const

See next slide for details

EXERCISE 1A

1. The member names and types can be read off the UML. Remember to use the exact names and types.
2. The constructor is straight forward
3. to_string: The to string method should return exactly the string name price<rounded to two decimals>. That is, if name="Apple" and price=2.476 the result should be Apple 2.48. Se example and hints below

```
item it={2,"Apple",3.7657};
it.to_string// Apple 3.77
item it={2,"Apple",3};
it.to_string// Apple 3.00
```

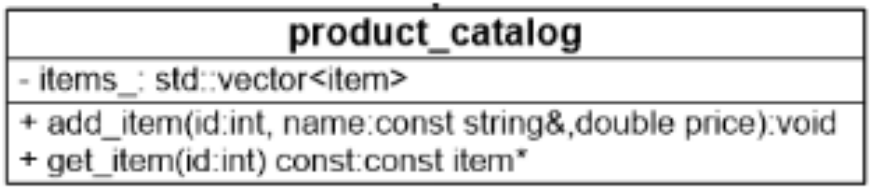
For rounding use the C++ 20 feature

```
std::format("{:.2f}", price)
```

which is available in header <format>

EXERCISE 1B

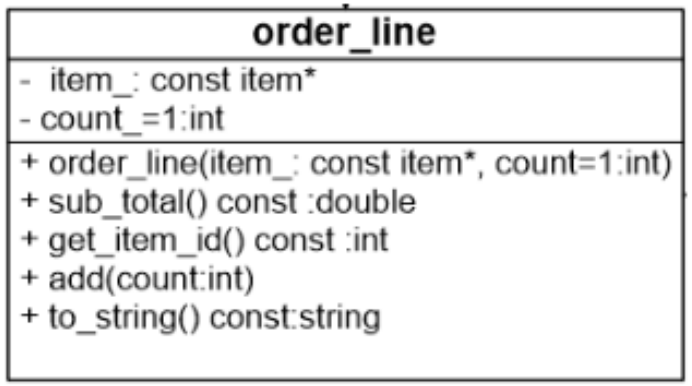
In this sub-exercise we implement as a class



1. std::vector<item> items_: The composition is realized by a vector of items.
2. void add_item(int,const std::string&,double): the method add item constructs the new item in the vector
3. const item* get_item(int) const: The method get item returns a const pointer to the item with the given id. If no element is found in the catalog a null pointer should be returned. One should notice that error handling of this null pointer in future exercises is not required, to limit the scope of the exercises.

EXERCISE 1 C

In this sub-exercise we implement class



The order_line class is in **association** with the item struct.

It is **not** a composition because it has no control over the item lifecycle, which the product_catalog has. The association is implemented via a **const pointer** to an item. const since order lines should not alter items.

EXERCISE 1 C

The order-line represents a line in an **order**(is added later) referring to the relevant item and an associated count that counts the number of this specific item

1. `const item *item_` : Is a const pointer to one of the items in the product catalog
2. `int count_` : Is the number of this specific item in the order.
3. `order_line(const item*,int count=1)` : The constructor initializes with the item pointer and a count that can be manually input otherwise it defaults to 1.
4. `double sub_total() const` : The method sub-total simply calculates the number of items times the item price.
5. `int get_item_id()` : This method retrieves the item id from the referred item.
6. `void add(int count)` : This method adds additional count to the order line if more items of the same type are purchased.
7. `std::string to_string() const` : The to string method should return string specified below **exactly**.

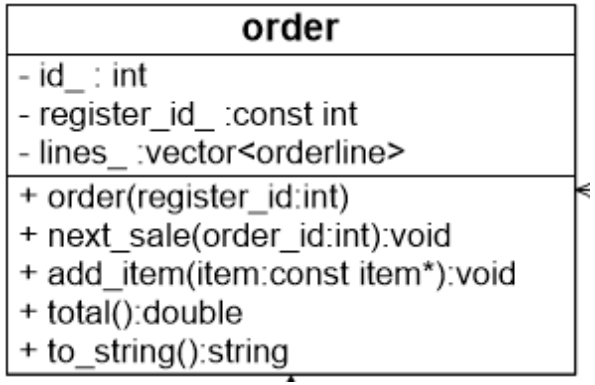
The to string method should return exactly this string:

If the item name is say **Apple** with **price 4** and the **count is 5**

The to string method should return **Apple 5 20.00** where the sub-total is rounded to two decimals

EXERCISE 1 D

Now implement order



1. `const int id_` : The id is the order id and is const since it does not change
2. `const int register_id_` : The register id is the id of the `checkout_counter` that is in relation as composition to the order. This member is going to be set by the `checkout_counter` in the next exercise.
3. `std::vector<order_line> lines_` : This vector establishes the composition relation with `order_lines` .
4. `void next_sale(int order_id)` : This method is meant to start a new sale. Since the order is going to be in composition (for technical reasons) with the class `checkout_counter` the `lines` must be cleared when a new sale is started. And also the order id must be updated when a new sale is started. This is the reason for the parameter `order_id` .
5. `void add_item(const item *item, int count = 1)` : This method adds a new item to the order. If an order line with this item already is present in the order the order-line must be updated with the count, otherwise a new order-line object should be added to the lines with the specified item and count
6. `double total() const` : This method simply calculates the total of the order which is the sum of sub-totals of each order-line.
7. `std::string to_string() const` : The to string method is the method that generates the bill of the order. This should be implemented with the constraints specified below to pass the test.

EXERCISE 1 D

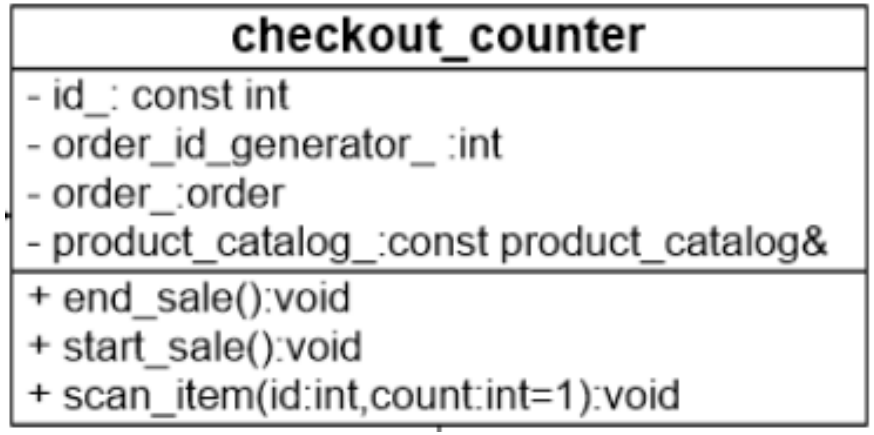
The `to_string` method should return a string **containing:**

1. **register:1** if the `register_id` is equal to 1
2. **order id_:1** if the `order_id` is equal to 1
3. For all order lines it should contain output of the to string method of order line which is **Apple 5 15.00** where **Apple** is the item name **5** is the **count** on the order line and **15.00** is the subtotal of the order line formatted to two decimals.

EXERCISE 1 E

CheckOutCounter

1. `id_: const int` :The check_out_counter has a private member `id_` of type `const int` . It is immutable
2. `order_id_generator_: int` : The check_out_counter has a private member `order_id_generator_` of type `int` which is an integer for generating the next id when a new sale is started
3. `order_:order` The check_out_counter has a member `order_` of type `order` . It has a permanent order object by composition, and why is that? Why do we not create a new order in the `start_sale` method. This is because we have not discussed dynamic memory allocation yet. So, rather artificially we need to reuse the same `order` for every new sale. This means that a **new sale should clear the old order**.



See next slide

EXERCISE 1 E

4. `product_catalog_: const product_catalog&` : This association to product catalog is implemented by a reference and not a pointer. This is viable since the relation is fixed for the entire lifetime of the application. The product catalog does not change. So here is an example of modelling an association by a reference.
5. `start_sale: void` : This method starts a new order and since we are forced to reuse the order it must be cleared and have a new id. This is why the order has a next sale method which takes a new order id (and clears the order). The new order id is generated by the member `order_id_counter` which when used for a new sale must be increased by **one!**.
6. `scan_item(id:int,count=1:int)` : This method is for scanning an item and setting manually a count of the specific item. The count has **default value 1** such that a simple scan simply counts 1. This method must consult the product catalog to retrieve the item and add the item and count to the order.
7. `end_sale():void` : This method simply produces the `to_string` value of the order and the sale is considered completed. starting a new sale is done by first invoking `start_sale()` and the process starts over.

EXERCISE 1 F

Try to test the system in a main method

EXERCISE 2-3

The purpose of the next two exercises is to get a feel for the difference between association and composition.

In exercise 2 we try to implement mainly by association and exercise 3 is mainly focused on using composition where it is possible

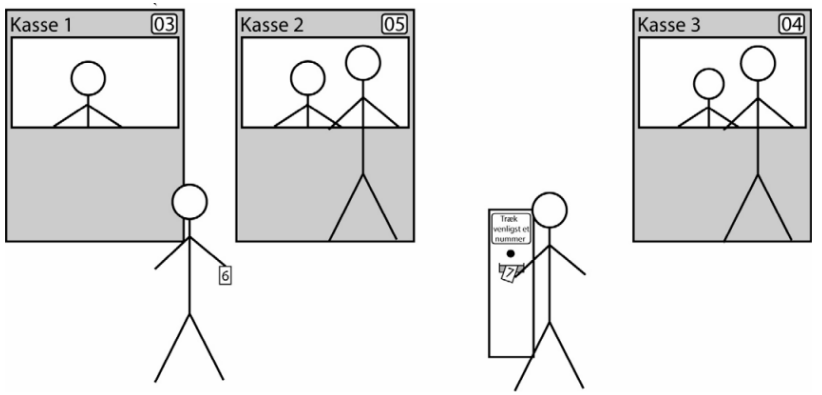
THE PROBLEM

We imagine that we are to implement a rudimentary Service queue management system.

That is, we imagine a setup like

1. A maximum of three counters
2. A number display and a button for each counter
3. A service-number dispenser with a button for next number

This could physically look like



PREPARED CODE

To make it feasible as an exercise the exercise comes with some prepared code for windows and for mac

For windows, the following components are prepared

1. Cursor.h and Cursor. Cpp which is code that makes it possible to move the point of output in a terminal
2. Display.h and Display.cpp is code for displaying “Tellers/Counters/Buy-station “
3. Main.cpp which holds code for react on key inputs in a switch

Explanation follows

WINDOWS

The following code is for windows

CURSOR

The Cursor files are depicted below, and the sole purpose is to move the “cursor”(output location of cout) in the terminal/console. And it works on WINDOWS

```cpp
#pragma once

#include <string>
#include "windows.h"

class Cursor {
public:
    Cursor();
    void cursorToXY(short x, short y) const;
private:
    HANDLE handleToConsoleWindow_;
};
```

```cpp
#include "Cursor.h"

Cursor::Cursor() {
    handleToConsoleWindow_ = GetStdHandle( nStdHandle: STD_OUTPUT_HANDLE);
}
// Upper left corner is (0,0).
// x-axis is horizontal left to right. y-axis is vertical top to bottom
void Cursor::cursorToXY(const short x, const short y) const {
    COORD pos;
    pos.X = x;
    pos.Y = y;
    SetConsoleCursorPosition( hConsoleOutput: handleToConsoleWindow_,  dwCursorPosition: pos);
}
```

DISPLAY

The display code is used to create objects (1,2,3 max 3),that displays and represents up to three Counters/selling places/Tellers or what you would call it. Making three such objects would show

```
Counter 1   Counter 2   Counter 3
  99          99          99
```

And a method for updating the value which default is set to 99

```
class Display {
public:
    Display(short);
    void update(int) const;
private:
    const Cursor cursor;
    int id_number_; // valid values : 1, 2, 3
};
```

```
Display::Display(const short id_number) :
        id_number_((id_number > 0 && id_number < 4 ? id_number : 0)), cursor(Cursor{}) {
    cursor.cursorToXY( x: static_cast<short>(12 * id_number_ - 7),  y: 8);
    std::cout << "Counter " + std::to_string( val: id_number);
    update( number: 99);
}

void Display::update(const int number) const {
    cursor.cursorToXY( x: static_cast<short>(12 * id_number_ - 5),  y: 9);
    std::cout << (number < 10 ? "0" : "") << number << std::endl << std::endl;
}
```

MAIN

In the main method some setup is preprogrammed to read key inputs and display the possible key inputs

Where the important part is shown below. The getch() function which works on windows

```
cout << "Hit 'n' to draw a numb
cout << "Hit '1' for new custom
cout << "Hit '2' for new custom
cout << "Hit '3' for new custom
cout << "Hit 'q' to quit.\n\n\n

char key;
cout.flush();
do {
    key = _getch();
    cout << key;
    switch (key) {
        case '1':
```

FOR MAC

The following code is for mac

TERMINAL SETTINGS

This code in terminal_settings.h/cpp is to set up the terminal such that is does not echo key strokes and doesn't need return to read a key.

Also, it resets the terminal settings on exit

```
#pragma once

void restoreTerminal();

void disableEcho();

char getKeyPress();
```

```
#include <iostream>
#include <unistd.h>
#include <termios.h>
#include "terminal_settings.h"
struct termios oldt;
void restoreTerminal() {
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
}

void disableEcho() {
    tcgetattr(STDIN_FILENO, &oldt);
    struct termios newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    atexit(restoreTerminal); // Ensure restoration on program exit
}

char getKeyPress() {
    return getchar();
}
```

CURSOR

The cursor is to move the cursor in the terminal

```h
#pragma once

class Cursor {
public:
    void cursorToXY(short x, short y) const

};
```

```cpp
#include "Cursor.h"
#include <iostream>

// Upper left corner is (0,0).
// x-axis is horizontal left to right. y-axis is vertical top to bottom
void Cursor::cursorToXY(const short x, const short y) const {

        std::cout << "\033[" << y << ";" << x << "H" << std::flush;

}
```

DISPLAY

The display is to show the counters in the terminal which is no different from the windows version

MAIN

The main part has a small difference from windows. I must call the terminal setting on startup and use the get char method and at the end it should for safety call restore terminal

```
disableEcho();

cout << "Hit 'n' to draw a number." << endl;
cout << "Hit '1' for new customer at counter 1." << endl;
cout << "Hit '2' for new customer at counter 2." << endl;
cout << "Hit '3' for new customer at counter 3." << endl;
cout << "Hit 'q' to quit.\n\n\n\n\n\n";


char key;
cout.flush();
do {
    key = getchar();
    switch (key) {
```

```
} while (key != 'q' && key != 'Q');

restoreTerminal();
```

PREPARED CODE

With this prepared code you should be able to construct a simulation of a queue system

The classes that could be involved are(besides the terminal_settings on mac)

Display,cursor: which are given

number_dispenser: representing the dispenser that one pushes to get a que number

counter: representing thesales place that shows the current serviced customer and which by a push method can ask for next customer in line from the dispenser and update the display

And then a full implementation of the main function

IMPORTANT

Whether it is window or mac the application should be run in the terminal since the clion run window does not exactly react as a terminal

WINDOWS! On windows it is practical to build with static linkage

```
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -static-libgcc -static-libstdc++")
set(CMAKE_EXE_LINKER_FLAGS "${CMAKE_EXE_LINKER_FLAGS} -static")
```

In order to execute the exe.file

On mac this is not necessary since the libraries are already on terminal path

AARHUS
UNIVERSITY